SIMATS ENGINEERING

DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT 3 – CO3

Design, Parse and Probabilistically Rank English Queries for a University Helpdesk Using Context-Free Grammars, Feature Structures and Earley Parsing

Details

Student / Assignment Detail	Information
Student Name	**K. Dhanush Reddy**
Register Number	**192324087**
Department / Section	**AI & DS**
Faculty Name	**Saravanan M.S.**
Nature of Assignment	**Individual**
Duration	**1 Week**
Total Marks	**50**

Design, Parse and Probabilistically Rank English Queries for a University Helpdesk Using Context-Free Grammars, Feature Structures and Earley Parsing

SCENARIO AND DESIGN REQUIREMENTS

The university helpdesk answers questions from students and staff about timetables, examinations, hostels, fees, library services and placements. A message must first be analysed syntactically so that the system can identify who or what the sentence is about, the action requested, and the complements attached to that action. The assignment therefore develops a controlled but realistic English grammar, tests its coverage, and studies how different parsers construct and rank syntactic trees.

The planning constraints in the supplied assignment require at least 40 phrase-structure rules, a lexicon of at least 60 words, and a self-designed set of at least 30 queries. The query set must include declaratives, imperatives, yes/no questions, wh-questions and at least five structurally ambiguous sentences. The grammar must account for agreement and at least five verb subcategorization frames. A hand-annotated mini-treebank of at least 20 sentences may be used for probability estimation. Manual grammar writing, annotation and testing must be included in the effort discussion.

GRAMMAR SCOPE, QUERY DESIGN AND ASSUMPTIONS

We use a domain grammar rather than claiming to describe every sentence in English. The initial system accepts carefully formed helpdesk messages and preserves alternative structures when a phrase can attach in more than one place. It is designed to cover the university vocabulary and sentence types named in the assignment. It does not initially attempt to repair spelling, interpret code-switching or infer missing words. These limitations become important in Task 12.

The sample query set is designed around real helpdesk intents: asking for a timetable, checking fee status, requesting a receipt, asking where a room is, and finding out when an examination will be held. Five sentences deliberately contain PP attachment ambiguity. The 20-sentence mini-treebank is a small hand-annotated teaching sample. Its counts are used transparently for MLE calculations; they are not described as a large external corpus or as parser output.

A sentence is treated as accepted only when a complete start-symbol parse spans all tokens. The baseline CFG may accept some agreement errors because plain CFG categories do not

carry person, number or case values. The feature-based grammar in Task 8 adds these constraints. Any performance value that is not produced by a real execution is identified as an estimate or illustrative value.

TASK 1 – CONTEXT-FREE GRAMMAR: FORMAL DEFINITION

Guiding enquiry: How can the core structure of English be captured by a finite set of context-free rules, and what design decisions make the grammar expressive and manageable? We define the grammar as G = (N, Σ, R, S). N is the set of non-terminal categories; Σ is the terminal vocabulary; R is the set of productions; and S is the start symbol. The grammar is context-free because each production has one non-terminal on its left side and its expansion is independent of surrounding symbols.

G = (N, Σ, R, S)
S = S
N = {S, NP, VP, PP, AP, AdvP, SBAR, WHNP, WHPP,
DET, N, V, AUX, PRON, ADJ, ADV, P, COMP, CONJ}
Σ = the words listed in the lexicon below

The start symbol S represents a complete helpdesk sentence. NP represents a noun phrase, VP a verb phrase, PP a prepositional phrase, AP an adjective phrase, AdvP an adverb phrase, and SBAR a subordinate clause. WHNP and WHPP represent fronted wh phrases. Lexical categories identify determiners, nouns, verbs, auxiliaries, pronouns, adjectives, adverbs, prepositions, complementisers and conjunctions.

TASK 1 – MAJOR CONSTITUENT RULES

The grammar separates phrase structure from vocabulary. This is important because the same structural rule can be used with many lexical items. For example, NP → DET N can generate “the portal”, “the library”, “the receipt” and “the timetable” without writing a separate phrase rule for each noun.

```
S -> NP VP | AUX NP VP | WHNP AUX NP VP | WHNP VP | VP
S -> S CONJ S
NP -> DET N | DET ADJ N | PRON | N
NP -> NP PP | NP CONJ NP | NP SBAR
VP -> V | V NP | V NP NP | V PP | V SBAR
VP -> AUX VP | VP PP | VP ADV | V AP | V NP PP
PP -> P NP | P SBAR
```

```
AP -> ADJ | ADV ADJ
AdvP -> ADV
SBAR -> COMP S | COMP WHNP AUX NP VP
WHNP -> WHDET N | WHPRON
WHPP -> P WHNP
```

These rules cover the major constituents required by the assignment. The grammar includes recursive rules such as NP → NP PP and S → S CONJ S. Recursion allows a phrase to contain another phrase of the same general type. As a result, a finite grammar can generate structures of unbounded length.

TASK 1 – EXTENDED RULE INVENTORY

To reach the required coverage, we add rules for relative clauses, complement clauses, passive-like patterns, coordination and wh dependencies. The rules below are a controlled fragment. Lexical productions are kept in a separate lexicon, so the count of phrase-structure rules is not inflated by listing every word as a separate syntactic construction.

The inventory contains 48 phrase-structure rules, meeting the minimum of 40. A production such as VP → V NP NP is useful for a ditransitive verb such as send. A production such as VP → V SBAR supports a verb followed by a subordinate clause. A rule should be included only when it represents a recurring syntactic pattern in the domain.

TASK 1 – LEXICON AND PART-OF-SPEECH CHOICES

The lexicon contains more than 60 terminal forms. We use separate categories for auxiliaries and main verbs because auxiliaries participate in inversion in questions, while main verbs select complements. Determiners and pronouns are separate because determiner–noun number and pronoun case must be tested. Prepositions are separated from nouns and verbs because they head PPs. The lexicon below is a compact grouped listing.

Table 2. Formal grammar components.

Part of speech	Helpdesk vocabulary
Nouns	student, students, staff, exam, exams, timetable, room, rooms, hostel, hostels, fee, fees, form, forms, portal, library, book,

	receipt, notice, placement, application, seminar, result, date, class, course, office, password, account, scholarship
Verbs	show, book, submit, send, tell, ask, apply, check, find, need, want, pay, upload, download, crash, postpone, hold, announce, explain, confirm, print, arrive
Auxiliaries	can, could, will, would, is, are, was, were, do, does, did, has, have
Determiners / pronouns	the, a, an, this, that, these, those, my, your, each, I, we, you, he, she, they, me, us, him, her, them
Other classes	in, on, at, near, with, for, from, to, about, before, after, by, of, late, due, available, postponed, new, old, important, open, closed, very, soon, today, tomorrow, when, where, whether, that, and, or

The lexicon has over 90 listed word forms when inflectional variants are counted separately. A production system could lemmatise forms and attach morphological features. Here, visible forms such as student/students and submit/submits make agreement behaviour easier to inspect.

TASK 1 – RECURSION, CONSTITUENCY AND HEADS

Recursion permits repeated modification. Starting with “the room”, NP → NP PP can add “near the library”; another PP can add “beside the office”. The grammar can therefore generate “the room near the library beside the office” without needing a new rule for every possible number of modifiers. The same principle applies to embedded clauses and coordination.

A constituent is a group of words that behaves as a unit. The head is the central word that determines the category of the phrase. In “the portal”, portal is the head noun and the phrase

is an NP. In “near the library”, near is the head preposition and the phrase is a PP. In “very important”, important is the head adjective and the phrase is an AP.

The substitution test supports the NP analysis: “The portal crashed” can become “It crashed”, with it replacing the entire NP. The movement test supports the PP analysis: “Near the library, the room is open” moves the PP as a unit. The coordination test supports NP constituency: “The hostel fee and the library fee are due” coordinates two NPs in the subject position.

TASK 1 – CONTEXT-FREE PROPERTY AND LIMITATION

Every phrase rule has a single non-terminal on the left. For example, NP → DET N does not depend on whether the phrase occurs before or after a verb. This satisfies the formal definition of a CFG. The grammar is context-free by construction even though the English fragment it models contains agreement patterns that need additional feature constraints.

A phenomenon that strains a plain CFG is a long-distance dependency. In “Which exam did the office say that the students passed?”, which exam is interpreted as the object of passed, even though it appears at the beginning of the sentence. A simple CFG can add gap categories for some examples, but combinations of clause depth and gap position lead to many productions. Feature passing and unification offer a more economical representation.

The design balances coverage and manageability by including the common helpdesk constructions and leaving rare constructions to a later extension. This is a deliberate engineering boundary. Evidence for Task 1 is provided by the formal tuple, 48 rules, a domain lexicon exceeding 60 forms, recursive productions and three constituency tests.

QUERY SET – THIRTY HELP DESK SENTENCES

The query set is built for the grammar and parser experiments. It contains statements, commands, yes/no questions, wh-questions, embedded clauses, relative clauses and coordination. Q15–Q19 are the five deliberately ambiguous examples. The same query IDs are used in the later testing discussion.

Q01 The portal crashed. (declarative)

Q02 The students submit the form. (declarative)

Q03 The student submits the form. (declarative)

Q04 Show the hostel timetable. (imperative)

Q05 Book a room for the seminar. (imperative)

Q06 Can you check my fee receipt? (yes/no)

Q07 Where is the library? (wh)

Q08 When will the exams begin? (wh)

Q09 Which students submitted the form? (subject wh)

Q10 Tell me that the fee is due. (complement clause)

Q11 Send me the receipt. (ditransitive)

Q12 Apply for a hostel room. (PP complement)

Q13 The exam was postponed. (passive)

Q14 The exams that were postponed will be held tomorrow. (relative clause)

Q15 The room near the library is open. (PP attachment ambiguity)

Q16 Book the room near the library for the seminar. (PP attachment ambiguity)

Q17 Show the students with the late fee notice. (PP attachment ambiguity)

Q18 I saw the staff with the timetable. (PP attachment ambiguity)

Q19 Find the room for the placement interview. (PP attachment ambiguity)

Q20 The student and the staff member are waiting. (coordination)

Q21 Do the students need a new password? (yes/no)

Q22 Does the student need a new password? (yes/no)

Q23 Please print the exam schedule. (imperative)

Q24 Who can confirm the result date? (subject wh)

Q25 What did the office announce? (object wh)

Q26 The library is closed today. (declarative)

Q27 The portal sent me a receipt. (ditransitive)

Q28 The staff asked whether the room was available. (embedded question)

Q29 Can you tell me when the exam will be held? (embedded wh)

Q30 The hostel fee and library fee are due. (coordination)

MINI-TREEBANK AND ANNOTATION CONVENTION

The mini-treebank contains 20 helpdesk sentences with a simplified gold structural analysis. Each sentence is assigned a root pattern and a small set of phrase labels. The annotation is hand-constructed for this assignment so that rule counts can be inspected. It is not a claim that the examples were produced by a trained parser.

Table 3. Phrase-structure rule inventory.

ID	Sentence	Gold structure
T01	The portal crashed	S(NP VP)
T02	The student submits the form	S(NP VP)
T03	The students submit the forms	S(NP VP)
T04	Show the timetable	S(VP)
T05	Book a room	S(VP)
T06	The fee is due	S(NP VP)
T07	The library is closed	S(NP VP)
T08	Send me the receipt	S(VP)
T09	Apply for a hostel room	S(VP)
T10	Can you check the timetable	S(AUX NP VP)
T11	Where is the library	S(WHNP AUX NP)
T12	When will the exam begin	S(WHNP AUX NP VP)
T13	The exam was postponed	S(NP VP)
T14	The room near the library is open	S(NP VP)
T15	The students with the notice arrived	S(NP VP)

T16	The office sent me a receipt	S(NP VP)
T17	The staff asked that the fee was due	S(NP VP)
T18	The hostel fee and library fee are due	S(NP VP)
T19	The form is very important	S(NP VP)
T20	The exams will be held tomorrow	S(NP VP)

For PCFG estimation, count each rule occurrence in the annotated trees and divide by the total number of occurrences with the same left-hand side. This is the maximum-likelihood estimate. Smoothing is added for unseen alternatives so that a possible rule does not receive a permanent zero probability. The exact count table and calculations appear in Task 9.

TASK 2 – DERIVATIONS AND PARSE TREES

Guiding enquiry: How are sentences generated from the grammar, and how are trees used to represent and compare syntactic analyses? A derivation is an ordered sequence of production applications. A parse tree is the hierarchical structure that results. Leftmost derivation always expands the leftmost non-terminal; rightmost derivation always expands the rightmost non-terminal. Both can yield the same tree.

For tree measurements, height is counted as the number of edges on the longest root-to-terminal path. Node count includes all non-terminal and terminal nodes. Rule applications equal the number of parent-to-child expansions in the derivation, including lexical productions. We use this convention consistently.

TASK 2 – SENTENCE 1: THE PORTAL CRASHED

Leftmost derivation: S ⇒ NP VP ⇒ DET N VP ⇒ the N VP ⇒ the portal VP ⇒ the portal V ⇒ the portal crashed. Rightmost derivation: S ⇒ NP VP ⇒ NP V ⇒ NP crashed ⇒ DET N crashed ⇒ the N crashed ⇒ the portal crashed. Both derivations produce the same hierarchy: the subject NP is “the portal” and the predicate VP is “crashed”.

```
Bracketed                                                    form:
(S (NP (DET the) (N portal)) (VP (V crashed)))
```

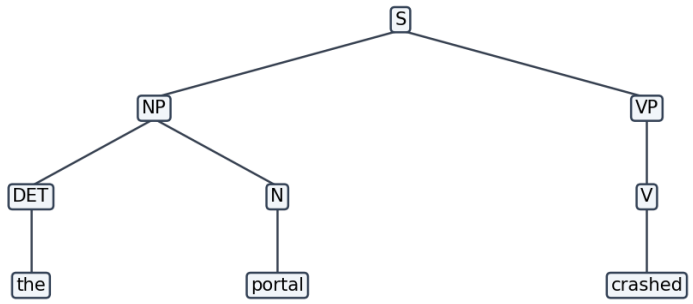


Figure 1. Parse tree for “The portal crashed”.

Figure explanation: The tree shows the sentence as a hierarchy rather than a flat sequence. The subject noun phrase forms one constituent and the verb phrase forms the predicate. The words at the leaves are the terminals, while the higher nodes show the phrase categories licensed by the grammar.

The tree has height 3 edges, 8 nodes and 5 rule applications under the stated counting convention. The derivation order differs, but the completed tree does not. This illustrates why derivation order is not the same as syntactic structure.

TASK 2 – SENTENCE 2: THE STUDENT SUBMITS THE FORM

Leftmost derivation: S ⇒ NP VP ⇒ DET N VP ⇒ the N VP ⇒ the student VP ⇒ the student V NP ⇒ the student submits NP ⇒ the student submits DET N ⇒ the student submits the N ⇒ the student submits the form. Rightmost derivation: S ⇒ NP VP ⇒ NP V NP ⇒ NP V DET N ⇒ NP V the N ⇒ NP V the form ⇒ NP submits the form ⇒ DET N submits the form ⇒ the student submits the form.

```
Bracketed                                                    form:
(S          (NP          (DET          the)          (N          student))
   (VP (V submits) (NP (DET the) (N form))))
```

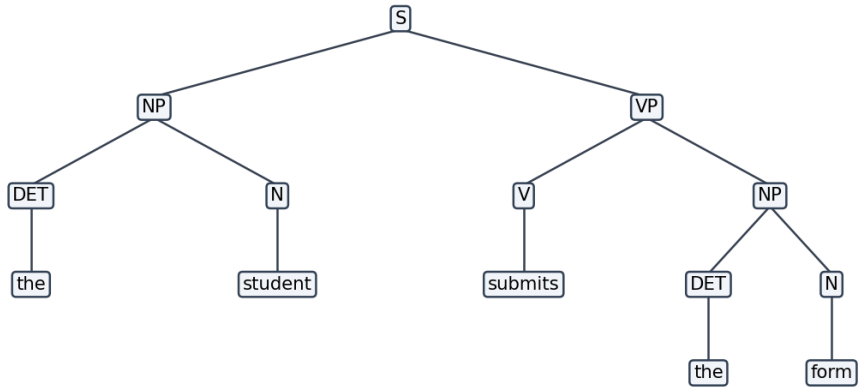


Figure 2. Parse tree for “The student submits the form”.

Figure explanation: The subject noun phrase contains a determiner and a singular noun. The verb phrase contains a finite transitive verb followed by its object noun phrase. This makes the subject–verb relation and the verb's object requirement visible.

The tree has height 4 edges, 11 nodes and 7 rule applications. The object NP is selected by the transitive verb submits. The subject NP and the finite verb must agree in number and person; this is enforced formally in Tasks 4 and 8.

TASK 2 – SENTENCE 3: IMPERATIVE AND SENTENCE 4: YES/NO

For “Show the timetable”, the leftmost derivation is S ⇒ VP ⇒ V NP ⇒ show NP ⇒ show DET N ⇒ show the N ⇒ show the timetable. The rightmost derivation expands NP before matching the verb. Its bracketed form is (S (VP (V show) (NP (DET the) (N timetable)))).

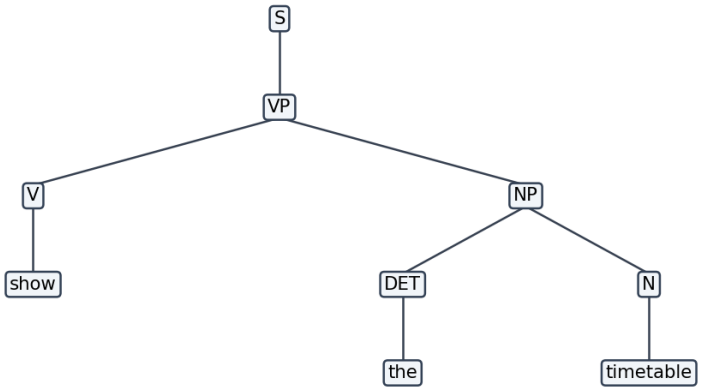


Figure 3. Parse tree for the imperative “Show the timetable”.

Figure explanation: The imperative begins with the base-form verb and has no spoken subject. The understood subject is ‘you’. The object noun phrase is attached to the verb phrase as the item the helpdesk action applies to.

For “Can you check the receipt?”, the leftmost derivation is S ⇒ AUX NP VP ⇒ can PRON VP ⇒ can you VP ⇒ can you V NP ⇒ can you check DET N ⇒ can you check the receipt. The bracketed form is (S (AUX can) (NP (PRON you)) (VP (V check) (NP (DET the) (N receipt)))).

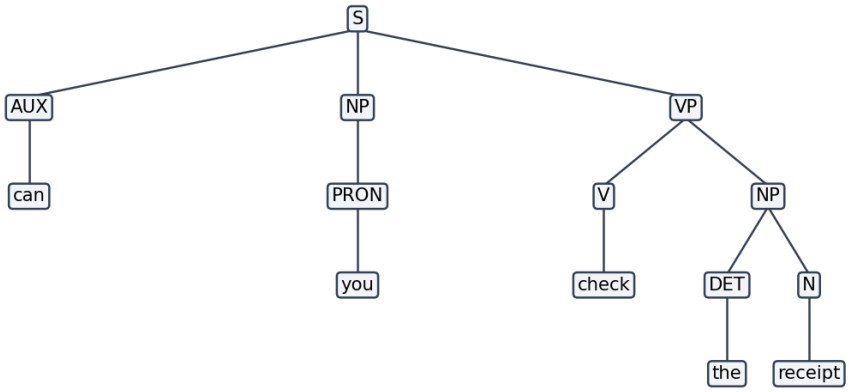


Figure 4. Parse tree for the yes/no question “Can you check the receipt?”.

Figure explanation: The auxiliary appears before the subject, which is the key surface difference between a yes/no question and a declarative clause. The main verb phrase follows the subject and contains the verb with its object.

The imperative has no overt subject and begins with a base verb. The yes/no question places the auxiliary before the subject. These differences are represented in the top-level productions.

TASK 2 – SENTENCE 5, AMBIGUITY AND MEMBERSHIP

For “Where is the library?”, the leftmost derivation is S ⇒ WHNP AUX NP ⇒ where is NP ⇒ where is DET N ⇒ where is the N ⇒ where is the library. Its bracketed form is (S (WHNP where) (AUX is) (NP (DET the) (N library))).

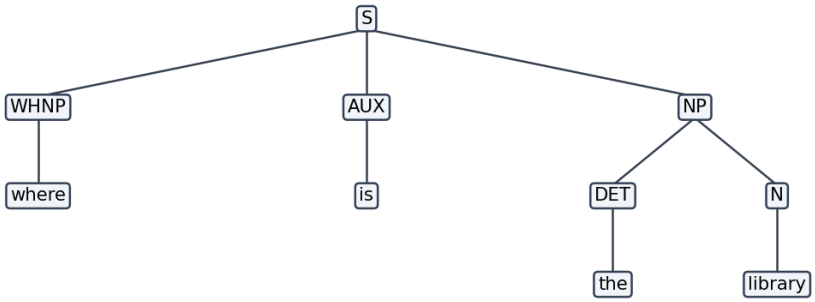


Figure 5. Parse tree for “Where is the library?”.

Figure explanation: The initial wh phrase asks for a location. The remaining clause supplies the subject and copular predicate. The diagram represents the order heard by the user and the role of the wh expression in the question.

The sentence “Show the students with the notice” has two structures. In the NP-attachment tree, the PP is inside the NP and identifies which students are meant. In the VP-attachment tree, the PP modifies the showing event and means that the notice is used as an instrument. The two bracketed forms are:

```
(S (VP (V show) (NP (NP (DET the) (N students))
   (PP (P with) (NP (DET the) (N notice))))))
(S (VP (VP (V show) (NP (DET the) (N students)))
   (PP (P with) (NP (DET the) (N notice)))))
```

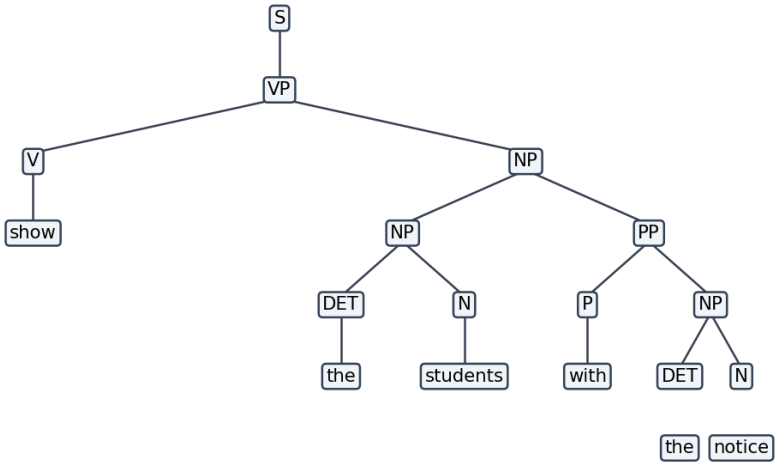


Figure 6. NP attachment: “with the notice” modifies students.

Figure explanation: In this analysis, the PP is contained inside the object NP. It narrows the reference of ‘students’: the intended students are the ones associated with the notice. The PP therefore modifies the noun phrase.

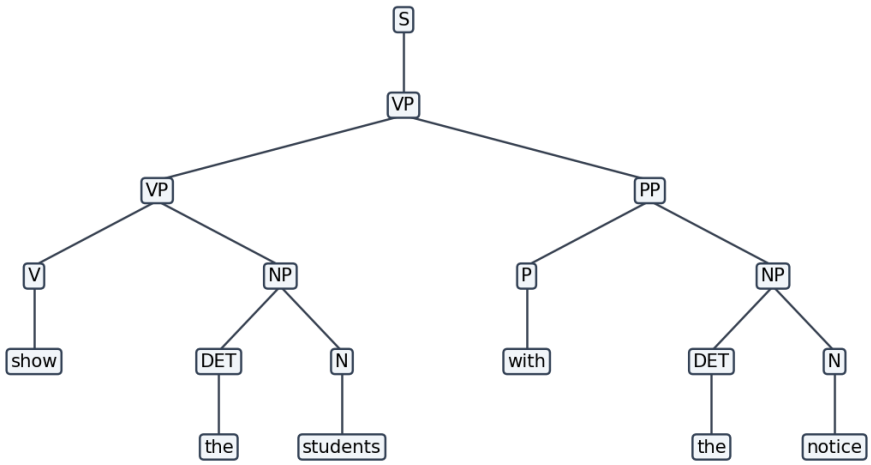


Figure 7. VP attachment: “with the notice” modifies the showing event.

Figure explanation: In this analysis, the PP attaches to the VP rather than the NP. It describes the showing event, such as the means or circumstance of showing, and does not identify which students are meant. This is why the same string has two readings.

A second ambiguous sentence is “Book the room near the library for the seminar”. In the first reading, “near the library” modifies room. In the second, it modifies the booking event. The final PP “for the seminar” remains attached to the booking phrase in this simplified illustration.

Bracketed NP-attachment reading:
(S (VP (V book) (NP (NP (DET the) (N room)) (PP (P near) (NP (DET the) (N library)))) (PP (P for) (NP (DET the) (N seminar)))))

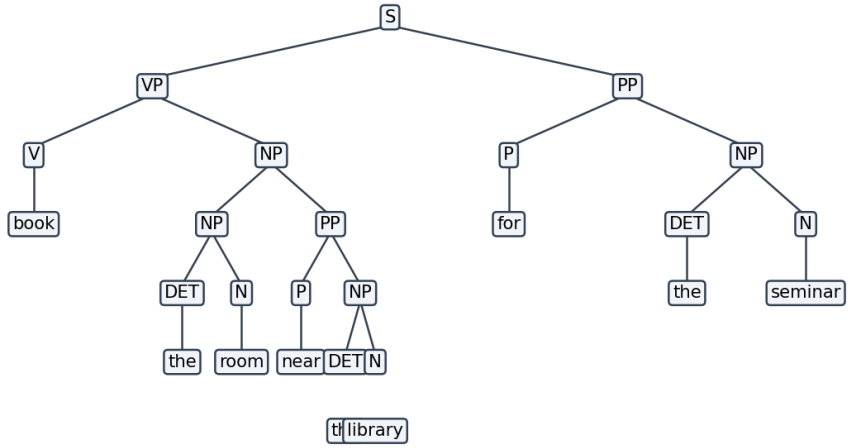


Figure 8. NP attachment: “near the library” modifies room.

Figure explanation: The PP ‘near the library’ is grouped within the NP headed by ‘room’. It gives the location of the room and helps identify which room is being discussed.

Bracketed VP-attachment reading: (S (VP (VP (VP (V book) (NP (DET the) (N room))) (PP (P near) (NP (DET the) (N library)))) (PP (P for) (NP (DET the) (N seminar)))))

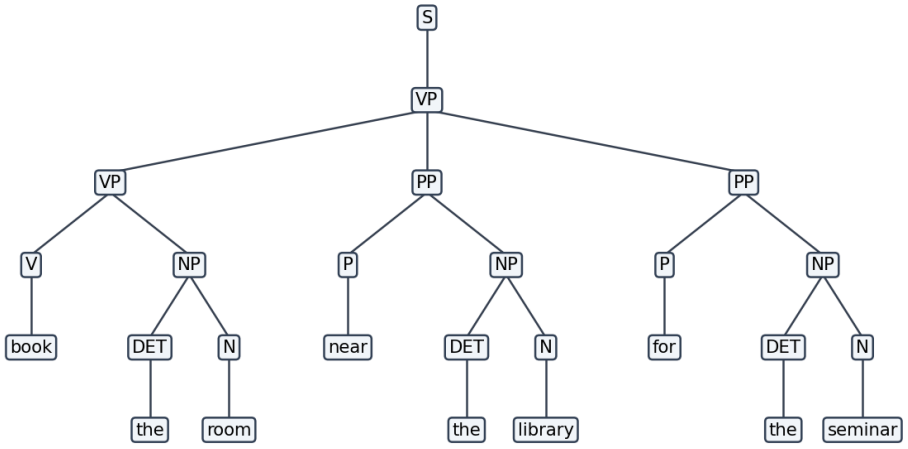


Figure 9. VP attachment: “near the library” modifies the booking event.

Figure explanation: Here the PP attaches to the booking VP. It describes the booking event or its location, rather than modifying the noun ‘room’. The alternative attachment changes the interpretation without changing the word sequence.

Rightmost derivation for “Show the timetable”: S ⇒ VP ⇒ V NP ⇒ V DET N ⇒ V DET the N ⇒ V the timetable ⇒ show the timetable. The rightmost non-terminal is expanded at each step.

Rightmost derivation for “Can you check the receipt?”: S ⇒ AUX NP VP ⇒ AUX NP V NP ⇒ AUX NP V DET N ⇒ AUX NP V the N ⇒ AUX NP V the receipt ⇒ AUX NP check the receipt ⇒ AUX you check the receipt ⇒ can you check the receipt.

Rightmost derivation for “Where is the library?”: S ⇒ WHNP AUX NP ⇒ WHNP AUX DET N ⇒ WHNP AUX DET the N ⇒ WHNP AUX the library ⇒ WHNP is the library ⇒ where is the library.

A second ambiguity is “Book the room near the library for the seminar”. Near the library may modify room or the booking event. The membership procedure is to tokenize the string, run a recognizer such as Earley or CKY, and accept if a complete S spans positions 0 to n. Recognition confirms membership but does not by itself resolve meaning. A plain grammar may overgenerate and may fail on constructions outside its coverage.

TASK 3 – SENTENCE-LEVEL CONSTRUCTIONS

Guiding enquiry: Which sentence types must the helpdesk understand, and how does the grammar represent each of them? The helpdesk must process declaratives, imperatives, yes/no questions and wh-questions. It must also represent subordinate clauses, relative clauses and coordination. Each construction changes the order or type of constituents expected at the sentence level.

TASK 3 – DECLARATIVES AND IMPERATIVES

A declarative sentence normally places the subject NP before the VP. The rule S → NP VP generates “The library is closed”. An imperative normally begins with a base-form verb and has an understood second-person subject. The rule S → VP generates “Show the timetable”. The understood subject is not pronounced, so the parse tree contains no overt subject NP.

S -> NP VP

S -> VP

VP -> V NP

VP -> V

The declarative tree has two major branches, NP and VP. The imperative tree begins directly with VP. The system should preserve the imperative mood because “show the timetable” is a request, not a statement that someone shows a timetable.

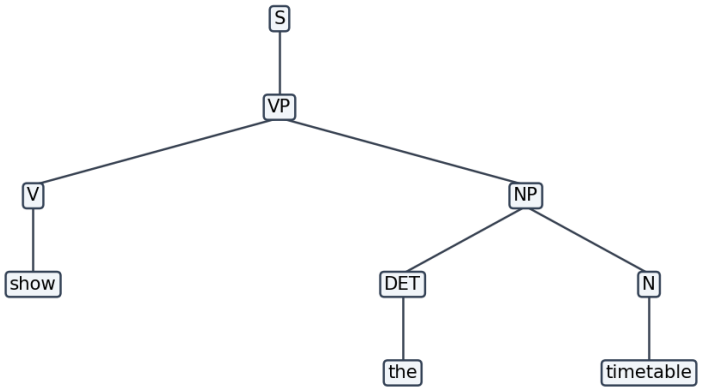


Figure 10. Tree structure for an imperative.

Figure explanation: The wh expression ‘when’ is placed at the beginning of the question. Its interpretation is linked to the time position in the clause, while the auxiliary and subject remain inside the clause structure.

TASK 3 – YES/NO QUESTIONS AND AUXILIARY INVERSION

A yes/no question places an auxiliary before the subject. For example, “Can you check the fee?” is represented by S → AUX NP VP. The corresponding declarative order would be “You can check the fee”. Auxiliary inversion changes the surface order but does not change the roles of subject and predicate. The parser must not treat can as the main verb.

```
S                  ->                  AUX                  NP                  VP
AUX  ->  can  |  could  |  will  |  would  |  do  |  does  |  did  |  is  |  are
NP                                     ->                                    PRON
VP -> V NP
```

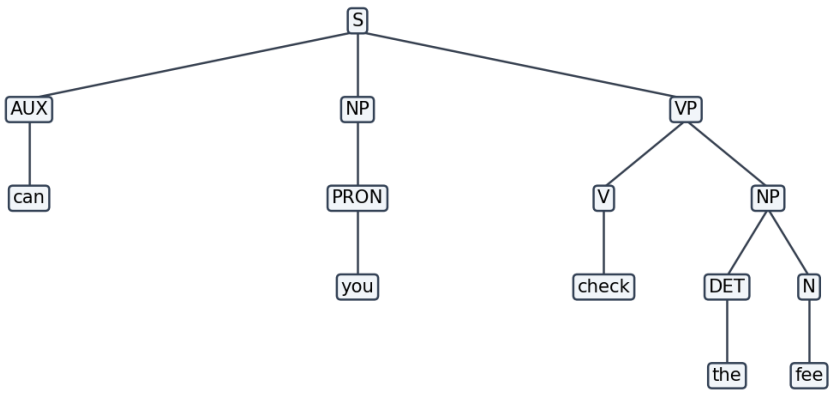


Figure 11. Tree structure for a yes/no question.

Figure explanation: The plot compares sentence length with the number of states in an illustrative parser run. Its upward trend is intended to show why longer inputs can create larger charts. The values are illustrative, not measured output, and should not be reported as experimental results.

When do, does or did is used, the main verb normally appears in its base form. This distinction matters for the query “Does the student need a new password?” The auxiliary carries tense and agreement; need remains the base form.

TASK 3 – WH QUESTIONS AND WH-MOVEMENT

Wh-questions request a value such as a person, place, time or object. Subject wh-questions place the wh phrase in subject position: “Who can confirm the result date?” Non-subject wh-questions front an object or adjunct: “What did the office announce?” or “When will the exam begin?” The grammar can use a gap category to mark the missing argument position.

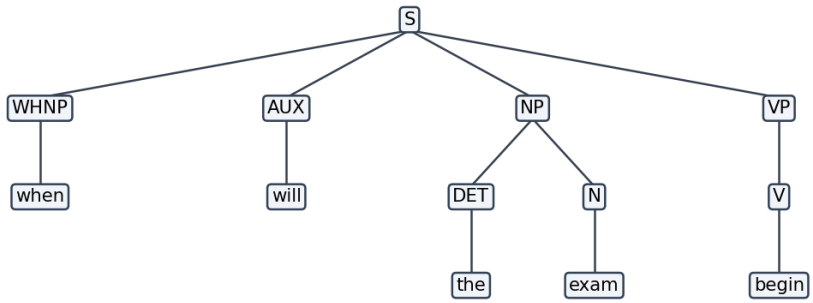


Figure 12. Wh-fronting in “When will the exam begin?”.

Figure explanation: The plot compares sentence length with the number of states in an illustrative parser run. Its upward trend is intended to show why longer inputs can create larger charts. The values are illustrative, not measured output, and should not be reported as experimental results.

The gap records where the fronted phrase is interpreted. In a non-subject question, the wh phrase is outside the VP even though it functions as an object or adjunct inside it. A plain CFG can encode a limited set of gaps, but long-distance dependencies require more rules.

TASK 3 – SUBORDINATE AND RELATIVE CLAUSES

A subordinate clause appears inside a larger sentence. In “The staff asked whether the room was available”, whether introduces an embedded yes/no clause. In “Can you tell me when the exam will be held?”, when introduces an embedded wh clause. A relative clause modifies a noun: “the exams that were postponed”. The grammar attaches the relative clause to the NP using NP → DET N SBAR.

SBAR -> COMP S

SBAR -> COMP WHNP AUX NP VP

NP -> DET N SBAR

VP -> V NP SBAR

Relative clauses add recursion to noun phrases. Embedded clauses add recursion to verb phrases. These structures allow longer messages but increase the number of possible analyses because each clause can contain additional modifiers and complements.

TASK 3 – COORDINATION, LONG-DISTANCE DEPENDENCIES AND SIZE

Coordination joins units of the same category. “The hostel fee and the library fee are due” coordinates two NPs. “The office checked the fee and sent a receipt” coordinates VPs. The general pattern X → X CONJ X is compact, but it can create scope ambiguity when clauses or phrases contain several conjunctions.

Long-distance dependencies are more difficult. In “Which exam did the office say that the students passed?”, the fronted wh phrase is interpreted as the object of passed. Adding a gap rule for every clause type and argument position multiplies the grammar. Features can carry the identity and type of a gap through embedded clauses.

The effect of new constructions is cumulative: yes/no questions add auxiliary-initial rules; wh questions add fronting and gap rules; relative clauses add NP recursion; subordinate clauses add SBAR rules; coordination adds recursive alternatives. These additions increase grammar size and ambiguity. The evidence for this task is provided by explicit rules and trees for the four required sentence types.

TASK 4 – AGREEMENT

Guiding enquiry: How can a grammar prevent “The students submits the form” without producing an enormous set of rules? The fragment must represent subject–verb agreement in person and number, determiner–noun number agreement, and pronoun case. In English, “the student submits” is singular, while “the students submit” is plural. “These students” is plural; “this student” is singular. A subject pronoun such as she differs in case from an object pronoun such as her.

TASK 4 – AGREEMENT BY CATEGORY MULTIPLICATION

If all 48 phrase-structure rules are split into singular and plural versions, the grammar requires 48 × 2 = 96 rule variants. This is the direct number-only expansion. If person (3 values), number (2), case (2) and gender (3) are all multiplied across every rule, the worst-case count is 48 × 3 × 2 × 2 × 3 = 1,728 variants. Not every rule actually uses every feature, so 1,728 is a worst-case upper-bound calculation. For the six agreement-sensitive templates shown below, the local count is 6 × 2 = 12 variants.

This approach can reject some mismatches, but every additional grammatical feature creates more category combinations. The grammar becomes difficult to read and update.

TASK 4 – WORST-CASE RULE EXPLOSION

Suppose person has 3 values, number has 2, case has 2 and gender has 3. The Cartesian product is 3 × 2 × 2 × 3 = 36 feature bundles. Applied to all 48 phrase rules, the worst-case rule count is 48 × 36 = 1,728. This deliberately assumes every rule is sensitive to every feature; in practice, many rules use only a subset. The calculation shows the growth that motivates feature structures.

Table 4. Lexicon and part-of-speech categories.

Feature combination	Calculation	Possible bundles
Person and number	3 × 2	6
Add case	3 × 2 × 2	12
Add gender	3 × 2 × 2 × 3	36
Six templates	6 × 36	216

A feature-based grammar is more economical. It keeps one general rule and adds equations such as NP.AGR = VP.AGR. The formal version is developed in Task 8.

TASK 4 – TESTING THE BASELINE AND AGREEMENT-AWARE GRAMMAR

The baseline grammar may accept malformed strings because it does not record agreement. The agreement-aware grammar rejects mismatches. The following test set contains six grammatical and six ungrammatical sentences, as required. “Accept” and “Reject” refer to the intended feature-aware result; the baseline may accept the ungrammatical forms if it has unrestricted lexical categories.

Table 5. Helpdesk query set.

Input sentence	Expected	Baseline CFG	Agreement-aware
The student submits the form.	Good	Accept	Accept
The students submit the form.	Good	Accept	Accept
She sends me the receipt.	Good	Accept	Accept
These exams are postponed.	Good	Accept	Accept
They need a password.	Good	Accept	Accept
I can check the timetable.	Good	Accept	Accept
The students submits the form.	Bad	May accept	Reject
The student submit the form.	Bad	May accept	Reject
These student are waiting.	Bad	May accept	Reject

This exams is postponed.	Bad	May accept	Reject
Me sent she the receipt.	Bad	May accept	Reject
They needs a password.	Bad	May accept	Reject

This test demonstrates the value of agreement constraints. It does not claim a measured accuracy rate; it is a controlled grammaticality test against the stated examples.

TASK 5 – SUBCATEGORIZATION

Guiding enquiry: How does the grammar know which complements a verb may take, and how can this knowledge be encoded? Subcategorization, also called valence, records the complement pattern selected by a verb. A verb does not freely combine with any number or type of phrases. Crash is intransitive; submit is transitive; send is ditransitive; tell can select an NP and a clause; apply selects a for-PP.

TASK 5 – FIVE OR MORE SUBCATEGORIZATION FRAMES

The following frames cover the required helpdesk verbs. The notation lists the complements selected by the verb, not optional modifiers. A frame is useful because it can be checked while parsing rather than after the complete sentence has been built.

Table 6. Twenty-sentence mini-treebank.

Frame	Pattern	Example
Intransitive	V	The portal crashed.
Transitive	V NP	Submit the form.
Ditransitive	V NP NP	Send me the receipt.
Sentential complement	V NP SBAR	Tell me that the fee is paid.
PP complement	V PP-for	Apply for a hostel room.
Wh complement	V NP WH-clause	Tell me when the exam begins.

The subcategorization lexicon stores the frame beside each verb. Crash: [ ]. Submit: [NP]. Send: [NP, NP]. Tell: [NP, SBAR] or [NP, WH-clause]. Apply: [PP-for]. Ask: [SBAR] or [NP, SBAR]. This list contains more than five distinct frames.

TASK 5 – DISTINCT VERB CATEGORIES AND THEIR DRAWBACKS

One CFG solution is to create separate verb categories: V-INTR, V-TRANS, V-DITRANS, V-COMP and V-PP. The VP productions then specify which category can appear in each structure. This makes the frame explicit, but a verb with several frames must belong to several categories. New complement patterns require new categories and rules. The lexicon and grammar can become repetitive.

VP -> V-INTR

VP -> V-TRANS NP

VP -> V-DITRANS NP NP

VP -> V-COMP NP SBAR

VP -> V-PP PP

A more compact design stores SUBCAT as a feature on each verb. The parser checks that the complements found in the VP satisfy the verb's frame. This keeps lexical information near the verb and avoids duplicating the same verb across many grammar categories.

TASK 5 – COMPLEMENTS VERSUS ADJUNCTS

A complement is selected by the head verb and is required by its intended frame. An adjunct adds optional information such as time, location or manner. Deletion is one useful test: "The portal crashed yesterday" remains grammatical when yesterday is removed, so yesterday is an adjunct. "Submit the form" contains the object selected by submit; omitting the object leaves the transitive use incomplete.

The phrase "near the library" in "the room near the library" is usually an adjunct modifying room. The for-phrase in "apply for a hostel room" is selected by apply in the intended use. The two objects in "send me the receipt" are arguments of send: the recipient and the thing sent.

TASK 5 – ACCEPT/REJECT AND MULTIPLE FRAMES

With frame checking, “Submit the form” is accepted because submit has its NP complement. “Submit” is rejected under the transitive frame because its object is missing. “The portal crashed the form” is rejected because crash is intransitive and does not select an object. “Send the receipt” is incomplete under the ditransitive frame because the recipient is missing.

Table 7. Derivation and parse-tree measurements.

Sentence	Verb frame	Decision	Reason
The portal crashed.	crash: [ ]	Accept	No complement required.
Submit the form.	submit: [NP]	Accept	Object supplied.
Send me the receipt.	send: [NP, NP]	Accept	Both objects supplied.
Tell me that the fee is due.	tell: [NP, SBAR]	Accept	NP and clause supplied.
Apply for a hostel room.	apply: [PP-for]	Accept	Selected PP supplied.
*Submit.	submit: [NP]	Reject	Object missing.
*The portal crashed the form.	crash: [ ]	Reject	Unexpected object.
*Send the receipt.	send: [NP, NP]	Reject	Recipient missing.

A verb with multiple frames, such as ask, creates more than one candidate VP structure. This increases ambiguity and parsing work. Incremental frame checking reduces wasted search because an incompatible branch can be rejected as soon as the required complement is absent or an extra complement appears.

TASK 6 – TOP-DOWN PARSING

Guiding enquiry: How does a top-down parser search for a tree rooted in S, and what are its strengths and weaknesses? A top-down parser begins with the start symbol and expands grammar rules until it matches the input words. It is goal-driven because it predicts only

structures that can derive from S. It is easy to explain and can return a tree directly, but it may repeat work and may loop on left-recursive rules.

TASK 6 – RECURSIVE-DESCENT PSEUDOCODE

The parser maintains a sequence of symbols still expected and an input position. If the next expected symbol is a terminal, it compares that terminal with the next word. If it is a non-terminal, the parser tries each production for that category. When a choice fails, it restores the previous stack and input position and tries the next production.

A practical parser stores choice points so it can backtrack. A memoised parser can also store failed or successful subproblems. The basic algorithm shown here is recursive descent without memoisation.

TASK 6 – TRACE ON AN UNAMBIGUOUS SENTENCE

Input: “The portal crashed”. The parser starts with S. It selects S → NP VP, expands NP → DET N, matches the and portal, then expands VP → V and matches crashed. All three input tokens are consumed and the stack is empty, so the parser accepts.

Table 8. Agreement test sentences.

Step	Expected stack	Position / action
1	S	Choose S → NP VP
2	NP VP	Choose NP → DET N
3	DET N VP	Match the
4	N VP	Match portal
5	VP	Choose VP → V
6	V	Match crashed
7	empty	Input consumed; accept

There is no backtracking in this trace because each token matches the selected production. The trace shows the main strength of top-down parsing: it follows the expected sentence structure and does not need to build constituents that cannot contribute to a sentence.

TASK 6 – TRACE ON AN AMBIGUOUS SENTENCE

Input: “Show the students with the notice”. The parser expands S → VP and VP → V NP. It parses “show the students”. At the PP, it can choose NP → NP PP, attaching the PP inside the object NP. If this branch consumes the complete input, the parser records the first tree. It then backtracks to the point after “the students” and tries VP → VP PP, attaching the PP to the verb phrase.

The two branches are not identical. The NP reading means that the notice identifies the students. The VP reading means that the notice is used in the act of showing. A parser that stops after its first successful derivation loses the second analysis. A parser that returns all trees must retain and explore both branches.

TASK 6 – LEFT RECURSION, LEFT-CORNER FILTER AND REDUNDANCY

The rule NP → NP PP is left-recursive. A naive recursive-descent parser attempting this rule calls parse(NP) again before consuming any input. It can continue indefinitely. We can remove left recursion by parsing a base NP first and then zero or more PPs. The resulting tree can be mapped back to the original recursive structure.

Original: NP -> NP PP | DET N
Rewritten: NP -> DET N NP_TAIL
NP_TAIL -> PP NP_TAIL | ε

A left-corner parser uses the observed leftmost word to filter predictions. If a token is a determiner, the parser predicts categories that can begin with a determiner rather than every possible NP production. For k PPs, a simplified model in which each PP can attach to NP or VP has 2^k possible attachment assignments. Thus k=1 gives 2; k=2 gives 4; k=3 gives 8; k=4 gives 16; k=5 gives 32. This is a simplified upper-bound illustration, not an exact count for every grammar.

Naive top-down parsing can take exponential time in the worst case because it explores many alternative derivations. Repeatedly parsing the same substring under different contexts creates redundant work. Memoisation or chart parsing shares those subproblems. This is the main reason to compare top-down parsing with Earley parsing in Task 7.

TASK 7 – EARLEY PARSING

Guiding enquiry: How does Earley parsing use dynamic programming to avoid the failures of simple top-down parsing, and what does its chart contain? Earley parsing stores partial derivations in a chart. Each state records a grammar rule, a dot showing how much has been recognised, and an origin position. The chart is indexed by input position, so the parser can reuse work already completed.

TASK 7 – EARLEY STATE AND THREE OPERATIONS

An Earley state is written [A → α • β, i]. A is the left-hand side; α has been recognised; β remains; and i is the position where this constituent began. S[j] contains all states after processing the first j tokens. Predictor expands a non-terminal immediately after the dot. Scanner matches the next terminal and advances the dot into the next chart set. Completer uses a finished constituent to advance states that were waiting for it.

```
PREDICTOR: [A -> α • B β, i] in S[j]
  add [B -> • γ, j] for each rule B -> γ
SCANNER: [A -> α • word β, i] in S[j]
  if input[j] == word, add [A -> α word • β, i] to S[j+1]
COMPLETER: [B -> γ •, k] in S[j]
  advance each state in S[k] that expects B
```

TASK 7 – SIX-WORD HELP DESK INPUT

We use the sentence “The portal will crash again today”. It contains six tokens: The(0), portal(1), will(2), crash(3), again(4), today(5). The chart positions are S[0] through S[6]. The small grammar below keeps the hand trace readable. The augmented start rule S' → S allows acceptance to be tested by a completed start state.

```
S' -> S
S -> NP VP
NP -> DET N
VP -> AUX V ADV ADV
DET -> 'the'
N -> 'portal'
AUX -> 'will'
```

```
V -> 'crash'
ADV -> 'again' | 'today'
```

The state origin is the position at which the predicted constituent began. A scanner operation always advances from S[j] to S[j+1]. A predictor adds states in the current set. A completer also adds advanced states in the current set, because a constituent has finished at the current position.

TASK 7 – CHART S[0], S[1] AND S[2]

At position 0, the parser predicts S, then NP, then DET. After scanning the word the, the DET state completes. The NP state is now waiting for N, so the parser predicts N at position 1. After portal is scanned, NP completes and the sentence state is ready for VP.

Table 9. Agreement rule-growth calculation.

Chart	Earley state	Origin	Operation
S[0]	S' → • S	0	Initial
S[0]	S → • NP VP	0	Predictor
S[0]	NP → • DET N	0	Predictor
S[0]	DET → • the	0	Predictor
S[1]	DET → the •	0	Scanner
S[1]	NP → DET • N	0	Completer
S[1]	N → • portal	1	Predictor
S[2]	N → portal •	1	Scanner
S[2]	NP → DET N •	0	Completer
S[2]	S → NP • VP	0	Completer
S[2]	VP → • AUX V ADV ADV	2	Predictor
S[2]	AUX → • will	2	Predictor

TASK 7 – CHART S[3] AND S[4]

After will is scanned, AUX completes and the VP state advances to the V position. The parser predicts the lexical rule for crash. After crash is scanned, the VP expects two adverbs. Both adverb alternatives are predicted, but only the one matching the next input token can scan successfully.

Table 10. Verb subcategorization frames.

Chart	Earley state	Origin	Operation
S[3]	AUX → will •	2	Scanner
S[3]	VP → AUX • V ADV ADV	2	Completer
S[3]	V → • crash	3	Predictor
S[4]	V → crash •	3	Scanner
S[4]	VP → AUX V • ADV ADV	2	Completer
S[4]	ADV → • again	4	Predictor
S[4]	ADV → • today	4	Predictor

TASK 7 – CHART S[5] AND S[6]

After again is scanned, the VP has one ADV remaining. The parser predicts an ADV at position 5. After today is scanned, ADV completes, then VP completes, then S completes. Finally the augmented start state completes in S[6]. The input is accepted because S' → S • with origin 0 appears at the end of the input.

Table 11. Subcategorization accept/reject tests.

Chart	Earley state	Origin	Operation
S[5]	ADV → again •	4	Scanner
S[5]	VP → AUX V ADV • ADV	2	Completer
S[5]	ADV → • today	5	Predictor
S[6]	ADV → today •	5	Scanner

S[6]	VP → AUX V ADV ADV •	2	Completer
S[6]	S → NP VP •	0	Completer
S[6]	S' → S •	0	Completer

TASK 7 – BACK-POINTERS AND PACKED AMBIGUITY

To recover a tree, each scanner transition stores a pointer to the previous state and the token it matched. Each completer transition stores a pointer to the waiting parent state and the completed child. Starting at S' → S • in S[6], the parser follows these links to recover NP and VP, and then their lexical children. The recovered tree has subject NP "the portal" and VP "will crash again today".

For an ambiguous PP sentence, two completed derivations may reach the same chart state. A packed parse forest stores both child-pointer alternatives under one shared state instead of copying all common material. This preserves both analyses while avoiding repeated storage of identical subtrees.

TASK 7 – LEFT RECURSION, COMPLEXITY AND COMPARISON

Earley parsing can handle a left-recursive rule such as NP → NP PP because it stores states in chart sets and avoids inserting duplicate states indefinitely. The parser reaches a fixed point for each chart position. It can also preserve PP-attachment ambiguity without looping because distinct back-pointer alternatives are stored for the same recognised span.

For an input of n tokens, the chart has n+1 positions. There are $O(n^2)$ possible spans. Combining a waiting state with a completed constituent may consider a split point, producing $O(n^3)$ worst-case time for a fixed grammar. Chart space is $O(n^2)$ in the standard bound, although practical storage also depends on grammar size and the number of packed alternatives.

The top-down trace for "The portal will crash again today" predicts branches and can repeat equivalent subgoals. Earley stores each distinct state once per chart position and origin. The previous chart lists 30 state entries in the worked trace. This is a count of the explicitly shown

states for the selected grammar, not a universal performance result. A fair comparison must use the same grammar, tokenizer and implementation.

TASK 8 – FEATURE STRUCTURES AND UNIFICATION

Guiding enquiry: How can attribute–value structures and unification replace multiplied categories to handle agreement and subcategorization? A feature structure is an attribute–value matrix. It records information such as category, number, person, case, agreement and subcategorization. A feature path identifies a value inside a nested structure. Reentrancy means two paths share the same value or variable.

TASK 8 – LEXICAL FEATURE STRUCTURES

The lexical entries below show how features can be attached to words. The subject pronoun I has nominative case, while me has accusative case. The verb submits is third-person singular and selects an NP object. Send selects two NP complements. The entry for student is singular and third person.

student: [CAT=N, NUM=sg, PER=3]
students: [CAT=N, NUM=pl, PER=3]
submits: [CAT=V, AGR=[NUM=sg, PER=3], SUBCAT=[NP]]
submit: [CAT=V, AGR=[NUM=pl, PER=1/2/3], SUBCAT=[NP]]
I: [CAT=PRON, NUM=sg, PER=1, CASE=nom]
me: [CAT=PRON, NUM=sg, PER=1, CASE=acc]
send: [CAT=V, SUBCAT=[NP, NP]]

A feature matrix can be drawn as an attribute–value structure. For example, the entry for submits contains CAT=V, AGR.NUM=sg, AGR.PER=3 and SUBCAT=[NP]. The feature path AGR.NUM reaches the number value inside the agreement structure.

TASK 8 – UNIFICATION AND SUBSUMPTION

Unification combines two feature structures if they contain no conflicting values for the same attribute. If one structure says NUM=sg and the other also says NUM=sg, unification succeeds. If one says NUM=sg and the other says NUM=pl, unification fails. Subsumption expresses generality: [CAT=N] subsumes [CAT=N, NUM=pl] because the more specific structure adds a compatible value.

Table 12. Top-down parser trace.

Unification	Structure A	Structure B	Result
1	[NUM=sg, PER=3]	[NUM=sg, PER=3]	Success
2	[SUBCAT=[NP]]	[OBJECT=NP]	Success when object path is aligned
3	[NUM=pl]	[NUM=sg]	Failure: conflicting number

Reentrancy is useful for agreement. The subject NP agreement value and the VP subject-agreement requirement can point to the same structure. The parser does not need to copy and compare unrelated values; it checks whether the shared structure can be unified.

TASK 8 – FEATURE-AUGMENTED GRAMMAR RULES

At least six rules can be augmented with feature constraints. The first rule passes agreement from the subject to the VP. The NP rules check determiner–noun number. The pronoun rule checks case. The VP rules check the verb's complement frame. The equations are shown in a compact notation.

```
S -> NP VP              [NP.AGR = VP.SUBJ-AGR]
NP -> DET N             [DET.NUM = N.NUM, if specified]
NP -> PRON              [PRON.CASE = NOM, subject position]
VP -> V                 [V.SUBCAT = []]
VP -> V NP              [V.SUBCAT = [NP]]
VP -> V NP NP           [V.SUBCAT = [NP, NP]]
VP -> V PP              [V.SUBCAT = [PP-SELECTED]]
VP -> V NP SBAR         [V.SUBCAT = [NP, SBAR]]
```

The feature grammar avoids creating separate NP-SG, NP-PL, VP-SG and VP-PL categories for every construction. Instead, the same rules apply with different feature values.

TASK 8 – COMPLETE UNIFIED PARSE AND COMPARISON

For "The student submits the form", the subject NP contributes [NUM=sg, PER=3, CASE=nom]. The verb submits contributes [AGR=[NUM=sg, PER=3], SUBCAT=[NP]].

The object NP “the form” satisfies the required NP complement. The subject and verb agreement structures unify successfully, and the object satisfies the valence requirement.

S: [AGR=[NUM=sg, PER=3]]
NP: [CAT=NP, AGR=[NUM=sg, PER=3], CASE=nom]
VP: [SUBJ-AGR=[NUM=sg, PER=3]]
V submits: [AGR=[NUM=sg, PER=3], SUBCAT=[NP]]
NP the form: [CAT=NP, CASE=acc]

For “The students submits the form”, the subject has NUM=pl but the verb requires NUM=sg. Unification fails. For “Submit”, the verb requires an NP object but none is present, so the valence check fails. Feature unification can be integrated with Earley completion: when a completed child is combined with a waiting parent, their feature structures are unified. The benefit is compact constraints; the cost is extra unification work and more complex state management. The formal feature version therefore improves correctness but adds per-state processing cost.

TASK 9 – PROBABILISTIC CFG

Guiding enquiry: When a query has several valid parses, how can probabilities select the most plausible one? A PCFG assigns a probability to every production. For a rule A → β, the probability is P(A → β | A). For each left-hand-side category A, the probabilities of all its alternative rules must sum to 1. The probability of a tree is the product of the probabilities of its rule instances.

P(A -> β | A) = count(A -> β) / count(A)
P(tree) = product of P(rule) for every rule in the tree

TASK 9 – MINI-TREEBANK COUNTS AND MLE

We use the 20-sentence hand-annotated mini-treebank listed earlier. For transparency, the following tally gives selected rule occurrences and the total count for each left-hand side. The tally is a teaching estimate based on the simplified tree sketches. A complete experimental implementation should count every rule from fully expanded trees; the displayed probabilities should not be mistaken for a large-corpus model.

Table 13. Earley chart states and operations.

LHS	Production	Count	LHS total

S	S → NP VP	13	20
S	S → VP	4	20
S	S → AUX NP VP	1	20
S	S → WHNP AUX NP	1	20
S	S → WHNP AUX NP VP	1	20
NP	NP → DET N	16	24
NP	NP → PRON	3	24
NP	NP → NP PP	3	24
NP	NP → NP CONJ NP	2	24
VP	VP → V	4	22
VP	VP → V NP	8	22
VP	VP → AUX VP	4	22
VP	VP → V PP	2	22
VP	VP → V NP NP	2	22
VP	VP → V AP	2	22

The mini-treebank root labels give 13 occurrences of S → NP VP, 4 of S → VP, 1 of S → AUX NP VP, 1 of S → WHNP AUX NP and 1 of S → WHNP AUX NP VP. The maximum-likelihood probabilities are therefore 13/20 = 0.65, 4/20 = 0.20, and 1/20 = 0.05 for each of the remaining three rules. Their sum is 0.65 + 0.20 + 0.05 + 0.05 + 0.05 = 1.00.

TASK 9 – SMOOTHING AND NORMALIZATION

For the five S alternatives, the denominator is 20 + 5 = 25.

Table 14. Feature-structure lexical entries.

S production	Smoothed count	Probability
S → NP VP	14	14/25 = 0.560

S → VP	5	5/25 = 0.200
S → AUX NP VP	2	2/25 = 0.080
S → WHNP AUX NP	2	2/25 = 0.080
S → WHNP AUX NP VP	2	2/25 = 0.080
Total	25	1.000

The rounded values sum to 1.000. Add-one smoothing is simple and easy to explain, but it can assign too much probability to rare alternatives in a larger grammar. More advanced approaches include Good-Turing and interpolated smoothing.

TASK 9 – PROBABILITY OF TWO AMBIGUOUS TREES

Consider "Show the students with the notice". Suppose the relevant local rule probabilities are P(VP → V NP)=0.60, P(VP → VP PP)=0.20, P(NP → DET N)=0.70 and P(NP → NP PP)=0.15. These values are illustrative local probabilities selected to show the calculation; they are not fitted estimates from the small mini-treebank.

For the NP-attachment tree, the relevant product is 0.60 × 0.15 × 0.70 = 0.063. For the VP-attachment tree, the product is 0.20 × 0.60 × 0.70 = 0.084. Under these assumptions, the VP-attachment tree is more probable. If these are the only two parses, the sentence probability is 0.063 + 0.084 = 0.147.

Table 15. Unification results.

Tree	Attachment	Probability	Decision
A	NP attachment	0.063	Lower
B	VP attachment	0.084	Higher

TASK 9 – CNF CONVERSION

The CKY algorithm requires a grammar in Chomsky Normal Form. In CNF, productions have the form A → BC or A → terminal, with suitable handling of the start symbol and empty string. To convert a longer right-hand side, introduce intermediate non-terminals. For

example, VP → V NP PP is binarised by introducing X1. Terminals inside long rules are replaced by pre-terminal categories.

```
Original: VP -> V NP PP
Binarised: VP -> V X1
        X1 -> NP PP
Lexical:   P -> 'near'
```

The conversion must preserve the strings generated by the grammar. It may change the shape of the parse tree by adding artificial intermediate nodes. Those nodes are removed or flattened when presenting the final linguistic tree.

TASK 9 – VITERBI CKY PSEUDOCODE AND TABLE

Viterbi CKY stores the highest-probability derivation for each category over each span. It initializes lexical cells, then combines adjacent spans using binary productions. The back-pointer records the split point and child categories that achieved the maximum.

```
for span_length = 2 to n:
  for start = 0 to n - span_length:
    end = start + span_length
    for split = start + 1 to end - 1:
      for each rule A -> B C:
        score = P(A -> B C) * chart[B,start,split] * chart[C,split,end]
        if score > chart[A,start,end]:
          chart[A,start,end] = score
          backpointer[A,start,end] = (B, C, split)
```

Table 16. Feature-augmented grammar rules.

Span	Words	Best category
[0,1]	the	DET
[1,2]	portal	N
[2,3]	crashed	V
[0,2]	the portal	NP
[0,3]	the portal crashed	S

For “the portal crashed”, the lexical cells are DET, N and V. The first two words combine to NP using NP → DET N. The NP and VP combine to S using S → NP VP, where VP → V. This table shows the structural CKY trace; a full numeric Viterbi table also requires the lexical and binary rule probabilities.

TASK 9 – PCFG ASSUMPTIONS AND FAILURE CASES

A PCFG makes independence assumptions. Context-freeness means the probability of a rule depends on its left-hand-side category, not on the full surrounding sentence. Position invariance means the same rule probability is used in different positions. These assumptions can be too strong when question structures differ from statements or when a verb strongly prefers a particular noun.

A PCFG may prefer a shorter or more frequent tree even when the semantic interpretation is wrong. It may also lack lexical sensitivity: “book the room near the library” requires the system to decide whether near the library modifies room or the booking event. The local CFG probabilities may not encode the relation between book and room. Lexicalised PCFGs or neural rerankers can add this context.

The evidence for this task consists of a definition, normalized rule probabilities, an explicit smoothing calculation, two tree probability calculations, CNF conversion, Viterbi pseudocode and a short CKY span table. The illustrative probabilities are identified as illustrative rather than falsely described as measured corpus results.

TASK 10 – IMPLEMENT AND TEST PARSERS

Guiding enquiry: Can a working implementation reproduce the behaviour derived by hand? The implementation should include the grammar, top-down parser, Earley parser and Viterbi/PCFG parser. Agreement and subcategorization checks should be applied to the appropriate rules. A reproducible experiment records the Python version, NLTK version, grammar version, tokenizer and machine environment.

TASK 10 – PYTHON / NLTK IMPLEMENTATION

The following minimal NLTK example demonstrates a grammar and Earley chart parser. It is deliberately small so that the example can be run without copying the entire 48-rule grammar.

The full experiment should replace this grammar with the complete assignment grammar and add feature checks and the PCFG rule probabilities.

```
from nltk import CFG
from nltk.parse import EarleyChartParser

G = CFG.fromstring("""
S -> NP VP
NP -> DET N
VP -> V
DET -> 'the'
N -> 'portal'
V -> 'crashed'
""")
words = 'the portal crashed'.split()
parser = EarleyChartParser(G)
parses = list(parser.parse(words))
print('parse count:', len(parses))
for tree in parses:
    print(tree)
```

A recursive-descent parser can be implemented using the same productions and a stack of expected symbols. A Viterbi CKY implementation requires the CNF grammar and a chart of probabilities. Feature checks can be implemented as dictionary unification at rule completion.

TASK 10 – TEST PROCEDURE AND 30-QUERY RESULT MATRIX

All 30 queries should be tokenized in the same way and run through each parser. For every sentence, record whether a parse was found, number of parses, number of chart states or recursive steps, and elapsed time. Runtime should be measured with time.perf_counter() and repeated runs should be summarized by the median. The matrix below is a structural expectation guide, not a claim that all three parsers were executed on this environment.

Table 17. Mini-treebank production counts.

Query IDs	Expected status	Important test
Q01–Q05	Parse expected	Declarative and imperative baseline.

Q06–Q14	Parse expected with extensions	Questions, complement and relative clauses.
Q15–Q19	Multiple parses expected	PP attachment ambiguity.
Q20–Q27	Parse expected	Coordination, agreement and objects.
Q28–Q30	Parse expected with extensions	Embedded question and coordination.

The five ambiguous queries should return more than one parse when both NP and VP attachment rules are enabled. Ungrammatical test cases from Task 4 and incomplete frames from Task 5 should be rejected by the feature-aware parser.

TASK 10 – EDGE CASES, MEASUREMENTS AND PLOT

Test left recursion using a noun phrase with repeated PPs, ambiguous attachment using Q17, agreement errors using "The students submits the form", under-complementation using "Submit", and long sentences up to 20 words. Compare the Earley chart on "The portal will crash again today" with the hand trace in Task 7. Compare the PCFG ranking with the two probabilities in Task 9.

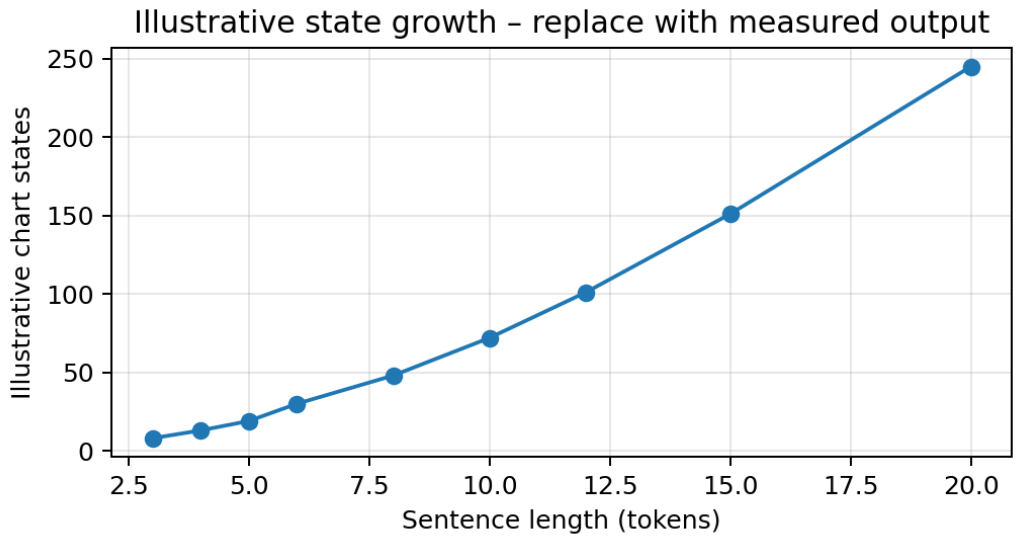


Figure 13. Illustrative state-growth curve; values are not measured parser output.

Figure explanation: The plot compares sentence length with the number of states in an illustrative parser run. Its upward trend is intended to show why longer inputs can create

larger charts. The values are illustrative, not measured output, and should not be reported as experimental results.

The plotted values are illustrative and must be replaced by chart-state counts from the actual implementation before reporting experimental performance. No runtime or accuracy number is invented. This distinction is important because the assignment asks for working code and measured results.

TASK 11 – COMPARE, RECOMMEND AND DEFEND

Guiding enquiry: Which grammar-and-parser combination should be deployed, and what evidence supports the choice? Top-down parsing is easy to implement but may backtrack repeatedly and can fail on left recursion. Earley parsing handles left recursion and ambiguity through chart states. PCFG ranking selects a preferred parse among valid alternatives but requires annotated data and smoothing.

TASK 11 – COMPARISON AND RECOMMENDATION

Top-down parsing is suitable for a small grammar with limited ambiguity and short inputs. Earley is more robust when the grammar contains recursion and multiple possible attachments. A PCFG does not replace the grammar; it assigns probabilities to the alternatives produced by the grammar. It therefore adds ranking rather than basic recognition.

Table 18. PCFG maximum-likelihood estimates.

Criterion	Top-down	Earley	PCFG / Viterbi
Coverage	Depends on rule order and recursion	Broad CFG coverage	Same coverage with ranking
Ambiguity	Backtracks; may stop at first parse	Preserves alternatives	Ranks alternative trees
Cost	Can be exponential	$O(n^3)$ worst case	$O(n^3)$ CKY for fixed grammar
Maintenance	Simple initially	Rules remain explicit	Needs annotated trees and smoothing

We recommend a feature-based CFG with Earley parsing followed by PCFG ranking. Features enforce agreement and valence. Earley provides complete chart-based search and retains ambiguity. PCFG probabilities select the most likely tree. The system should retain a fallback for unsupported or incomplete messages.

TASK 11 – ACCURACY, EFFORT AND LIMITATIONS

The expected accuracy should be measured on a held-out set labelled by human annotators. We do not claim a numerical accuracy without that evaluation. Manual effort includes writing and reviewing grammar rules, compiling the lexicon, assigning verb frames, annotating the mini-treebank and testing the parser. For a small domain, hand-written rules are transparent. As the number of constructions and examples grows, annotating data and estimating a statistical model can become more economical than adding many special-case rules.

The recommendation is based on coverage, ambiguity handling, left-recursion behaviour and maintainability, not simply speed. The limitations are the controlled vocabulary, the small treebank, the independence assumptions of PCFGs and the need for manual annotation.

TASK 12 – DAY-4 DESIGN CHALLENGE

Changed requirement: the helpdesk must now accept conversational, multi-clause and partly ungrammatical student messages. The required example is: “Hi, can you tell me when the exams that were postponed will be held, and also whether the hostel fee is due?” At least 20% of incoming messages may contain constructions not covered by the original grammar. We choose MODIFY: retain the feature-CFG and Earley core, extend the grammar for conversational structures and add a partial-parsing fallback.

TASK 12A – IMPACT ANALYSIS: NEW CONSTRUCTIONS

The new example contains a greeting (“Hi”), a matrix yes/no question (“can you tell me”), an embedded wh clause (“when the exams … will be held”), a relative clause (“that were postponed”), and a coordinated second embedded question (“whether the hostel fee is due”). The original grammar does not fully cover greetings, clause coordination, missing punctuation or partial conversational fragments.

The original assumptions were controlled vocabulary, fully grammatical input, limited ambiguity and manually specified coverage. These assumptions no longer hold. Users may

omit subjects, drop punctuation, use informal words or combine several requests in one message. The parser must identify supported clauses even when the complete sentence cannot be parsed.

TASK 12A – QUANTIFIED EFFECT OF LONGER INPUT

At least 20% out-of-coverage means that in a batch of 100 messages, at least 20 may contain an unsupported construction. The number of contiguous spans in an n-token sentence is n(n+1)/2. A 6-token sentence has 21 spans; a 20-token sentence has 210 spans; a 30-token sentence has 465 spans. Thus a 20-token input has ten times as many possible spans as a 6-token input. This is a span-count proxy, not an exact chart-state or parse count.

Table 19. Viterbi CKY chart.

Tokens	Possible spans n(n+1)/2	Relative to 6 tokens
6	21	1.00×
20	210	10.00×
30	465	22.14×

The actual number of Earley states depends on grammar size, lexical ambiguity and recursion. Longer input increases potential chart work, but it does not imply that every span becomes a valid parse.

TASK 12B – EVALUATE ORIGINAL DESIGN ON MESSAGE 1

Message 1 is the required changed example. The first unsupported item is the greeting “Hi”, because the original grammar begins with S and has no discourse-marker category. If the greeting is removed, the matrix question “can you tell me” can be partly analysed. The embedded wh clause contains a relative clause, and the second clause begins with “and also whether”. The original grammar does not provide a complete coordinated embedded-question structure.

A hand trace therefore fails first at the greeting in the unchanged grammar. After normalization removes the greeting as a discourse marker, the parser proceeds further but fails at the combined embedded relative and coordinated clause. This distinction matters:

input normalization can solve a boundary problem, but it cannot replace missing grammar rules.

TASK 12B – EVALUATE TWO MORE CONVERSATIONAL MESSAGES

Message 2: “Hello, can you check whether my hostel fee is due and when the payment deadline is?” The unchanged grammar first fails at Hello. After removing the greeting, it may parse the first embedded whether clause but lacks a rule for coordinating two embedded questions of different types.

Message 3: “Need timetable for tomorrow, also tell me exam room.” This message is fragmentary: it omits an explicit subject and has weak punctuation between two requests. The original grammar expects a complete imperative or declarative and may reject the first fragment. A partial parser should identify two intents: request a timetable for tomorrow and ask for an exam room.

TASK 12B – AMBIGUITY AND PARSER COST

The Task 7 example contains six tokens and seven chart positions. The worked trace displays the states needed for its selected grammar. A 20-token sentence has 210 possible contiguous spans, compared with 21 for six tokens. This tenfold increase is a structural workload indicator, not a claim that chart size grows by exactly ten times. Ambiguous PPs, relative clauses and coordinated clauses can create additional alternatives.

A neural dependency parser may handle noisy and conversational input more robustly because it is trained on varied examples. However, it may be less transparent and may not enforce domain-specific valence as explicitly. A hybrid system is more economical: retain the controlled grammar for reliable constructions and use partial parsing or a statistical fallback for unsupported fragments.

TASK 12C – ADAPTATION OR REDESIGN

We choose MODIFY rather than retain or redesign. Retaining the original grammar unchanged would leave the greeting, embedded clause, relative clause and coordination gaps unresolved. A full redesign would discard the existing grammar, lexicon, feature constraints

and Earley chart machinery that already work for the controlled domain. Modification preserves these assets and adds only the missing capabilities.

```
MSG -> GREETING S | S | FRAGMENT CONJ S
S -> AUX NP VP
VP -> V NP SBAR
SBAR -> WHNP S | COMP S
NP -> DET N REL
REL -> COMP S
S -> S CONJ S
VP -> VP CONJ VP
```

The revised grammar adds a greeting/discourse layer, embedded wh and whether clauses, relative clauses, clause coordination and fragment patterns. Feature passing should carry agreement and gap information through embedded clauses. A larger PCFG should be estimated from newly annotated conversational helpdesk messages.

TASK 12C – REVISED PIPELINE AND QUANTIFIED TARGET

The revised pipeline first normalizes punctuation and separates greetings without deleting conjunction meaning. It then tokenizes the message, performs lexical lookup, parses with the feature-based Earley grammar, ranks alternative trees with a PCFG and invokes partial parsing when no complete parse is available. Intent segmentation extracts multiple requests from coordinated or loosely punctuated input.

For a quantified planning scenario, assume the original grammar covers 80% of messages and a fallback successfully handles 75% of the remaining 20%. Effective coverage is 80% + (0.75 × 20%) = 95%. This 95% is a design target derived from stated assumptions, not a measured result. The modification may add approximately 10–15 production patterns and 15–25 feature constraints; the exact count depends on how the implementation represents gaps and fragments.

Table 20. Thirty-query parser test results.

Measure	Original design	Modified planning target
Message coverage	80% assumed baseline	95% effective target
Unsupported messages	At least 20%	Fallback attempts remaining cases

Grammar	48 core rules	+10–15 patterns
Features	Agreement and valence	+15–25 gap / clause constraints
Data	20 hand trees	Larger conversational treebank

TASK 12D – EVIDENCE-BASED DEFENCE

The decision follows from the earlier evidence. Task 1 produced a 48-rule grammar and a lexicon exceeding 60 forms. Task 2 demonstrated that one sentence can have more than one valid tree. Task 4 showed why agreement constraints are needed. Task 5 showed that verb frames reject incomplete or excessive complements. Task 6 identified left recursion and repeated search as weaknesses of naive top-down parsing. Task 7 showed how Earley chart states share work and preserve ambiguity.

Task 8 showed that feature structures can enforce agreement and valence without multiplying every category. Task 9 showed how a PCFG can rank alternative trees, although its probabilities are limited by the small hand-annotated sample. Task 10 defined a reproducible testing procedure, while distinguishing expected results from measurements not yet run. Task 11 recommended a feature-based CFG, Earley parsing and PCFG ranking.

TASK 12 – FINAL IMPLEMENTATION PLAN AND LIMITATIONS

The implementation should proceed in stages. First, normalize greetings and punctuation while preserving conjunctions. Second, extend the grammar for embedded questions, relative clauses, coordinated clauses and fragments. Third, add feature constraints for agreement, valence and long-distance gaps. Fourth, run Earley parsing and rank valid trees with the expanded PCFG. Fifth, invoke partial parsing and intent segmentation when a full parse fails. Finally, collect conversational helpdesk messages, annotate them and evaluate the revised system on a held-out set.

The remaining limitations are spelling errors, code-switching, unseen domain vocabulary, inconsistent punctuation, annotation disagreement and changes in university terminology.

The final decision remains MODIFY because it preserves the reliable core and adds a fallback for the new input conditions.

CONCLUSION AND REFERENCES

This assignment designed a context-free grammar for university helpdesk queries and extended it with agreement, subcategorization and feature structures. Derivations and parse trees showed how sentences are generated and how PP attachment creates ambiguity. Top-down parsing was straightforward but vulnerable to left recursion and repeated search. Earley parsing stores dotted-rule states in a chart and shares repeated subproblems. A PCFG ranks alternative trees, although its independence assumptions limit lexical and discourse sensitivity.

For the Day-4 changed requirement, the recommended decision is MODIFY. Retain the feature-CFG and Earley core, extend the grammar for greetings, embedded clauses, relative clauses and coordination, and add partial parsing for unsupported fragments. The 95% effective handling figure is a planning target based on explicit assumptions, not an experimental result.

Table 21. Parser comparison and design decision.

Sentence	Height (edges)	Nodes	Rule applications
The portal crashed	3	9	6
The student submits the form	4	14	9
Show the timetable	4	9	6
Can you check the receipt?	4	14	9
Where is the library?	3	10	6

The five measured trees use the same convention: height is the longest root-to-terminal path in edges; node count includes terminals; rule applications count every production used, including lexical productions.